
HỌC PHẦN NÂNG CAO VỚI C++

BỘ MÔN CÔNG NGHỆ THÔNG TIN

CHƯƠNG 5 – CƠ CHẾ ĐA HÌNH

BỘ MÔN CÔNG NGHỆ THÔNG TIN

- ❖ 5.1. Hàm ảo
- ❖ 5.2. Hàm thuần ảo
- ❖ 5.3. Lớp trừu tượng
- ❖ 5.4. Các thành viên ảo của một lớp

Hàm ảo

- ❖ Hàm ảo cho phép chương trình chọn đúng hàm của đối tượng thực sự đang dùng, chứ không phải dựa trên kiểu khai báo.
- ❖ Trong C++, khi ta có lớp cha và lớp con, thì lớp con có thể viết lại (ghi đè) hàm của lớp cha.
- ❖ Nhưng nếu ta gọi hàm qua con trỏ hoặc tham chiếu của lớp cha, C++ sẽ mặc định gọi hàm của lớp cha.
- ❖ Để C++ biết rằng: “*hãy gọi đúng hàm của lớp con*”, ta phải dùng **hàm ảo** (virtual function).

Hàm ảo

❖ Viết hàm ảo dành cho tính tiền trong đặt xe grab:

- Xe máy
- Xe oto

```
#include <iostream>
using namespace std;

// Lớp cha: Phương tiện Grab
class Grab {
public:
    virtual void datxe(int quangDuong) { // Hàm ảo
        cout << "Đang dat xe..." << endl;
    }
};

// GrabBike
class GrabBike : public Grab {
public:
    void datxe(int quangduong) override {
        int gia = quangduong * 8000; // 8k/km
        cout << "Dat GrabBike - Quang duong: " << quangduong
            << " km, Gia: " << gia << " VND" << endl;
    }
};

// GrabCar
class GrabCar : public Grab {
public:
    void datxe(int quangduong) override {
        int gia = quangduong * 15000; // 15k/km
        cout << "Dat GrabCar - Quang duong: " << quangduong
            << " km, Gia: " << gia << " VND" << endl;
    }
};
```

```
int main() {
    Grab* phuongtien; // Con trỏ Lớp cha

    GrabBike bike;
    GrabCar car;
    // Đặt GrabBike
    phuongtien = &bike;
    phuongtien->datxe(5); // 5 km
    // Đặt GrabCar
    phuongtien = &car;
    phuongtien->datxe(5); // 5 km

    return 0;
}
```

Hàm thuần ảo

- ❖ Hàm thuần ảo là hàm ảo đặc biệt: nó không có phần thân trong lớp cha, mà chỉ có khai báo.
- ❖ Lớp cha chỉ đưa ra “**khuôn mẫu**”, bắt buộc các lớp con phải viết lại.
 - `virtual void tenham() = 0;`
- ❖ Dấu `= 0` → báo hiệu đây là hàm thuần ảo.
- ❖ Lớp chứa hàm thuần ảo gọi là lớp trừu tượng (abstract class).
- ❖ Không thể tạo đối tượng trực tiếp từ lớp trừu tượng.
- ❖ Chỉ có thể tạo đối tượng từ lớp con, vì lớp con phải định nghĩa lại hàm đó.

```
#include <iostream>
using namespace std;

// Lớp trừu tượng (có hàm thuần ảo)
class Grab {
public:
    virtual int tinhhtien(int quangDuong) = 0; // Hàm thuần ảo
};

// GrabBike
class GrabBike : public Grab {
public:
    int tinhhtien(int quangduong) override {
        return quangduong * 8000; // 8k/km
    }
};

// GrabCar
class GrabCar : public Grab {
public:
    int tinhhtien(int quangduong) override {
        return quangduong * 15000; // 15k/km
    }
};

int main() {
    Grab* xe; // Con trỏ Lớp cha

    GrabBike bike;
    GrabCar car;

    xe = &bike;
    cout << "GrabBike 5km: " << xe->tinhhtien(5) << " VND" << endl;

    xe = &car;
    cout << "GrabCar 5km: " << xe->tinhhtien(5) << " VND" << endl;

    return 0;
}
```

- ❖ Lớp trừu tượng là gì?
 - Lớp trừu tượng là lớp mà:
 - Có ít nhất một hàm thuần ảo
 - Dùng để làm khuôn mẫu cho các lớp con.
 - Không thể tạo đối tượng trực tiếp từ lớp trừu tượng.
- ❖ Mục đích: ép các lớp con phải viết lại (override) các hàm mà lớp cha quy định. Hay có thể hiểu nôm na như sau: "Ai kế thừa tôi thì phải thực hiện những điều khoản này".

Lớp trừu tượng

- ❖ Khi nào dùng lớp trừu tượng?
 - Khi ta muốn các lớp con có hành vi chung, nhưng mỗi lớp con sẽ làm theo cách riêng.
 - Khi ta muốn ép buộc các lớp con phải cài đặt một số chức năng nhất định.
 - Khi ta cần đa hình: cùng một lệnh gọi → hành vi khác nhau.

Ví dụ - Lớp trừu tượng (20')

- ❖ Trong một trường đại học:
 - Giảng viên: có hoạt động là giảng dạy
 - Sinh viên: có hoạt động là học tập
 - Cán bộ: có hoạt động làm việc hành chính

Ví dụ - Lớp trừu tượng

```
#include <iostream>
using namespace std;

// Lớp trừu tượng
class Connguai {
public:
    virtual void hoatdong() = 0; // hàm thuần ảo
};

// Sinh viên
class SinhVien : public Connguai {
public:
    void hoatdong() override { // override để ghi đè hàm ảo trong dẫn xuất lớp con
        cout << "Sinh vien đang học tập." << endl;
    }
};

// Giảng viên
class GiangVien : public Connguai {
public:
    void hoatdong() override {
        cout << "Giang vien đang giảng dạy." << endl;
    }
};

// Nhân viên
class Canbo : public Connguai {
public:
    void hoatdong() override {
        cout << "Can bo đang làm việc hành chính." << endl;
    }
};
```

```
int main() {
    Connguai* ds[3];

    ds[0] = new SinhVien();
    ds[1] = new GiangVien();
    ds[2] = new Canbo();

    for (int i = 0; i < 3; i++) {
        ds[i]->hoatdong();
    }

    // Giải phóng bộ nhớ
    for (int i = 0; i < 3; i++) {
        delete ds[i];
    }

    return 0;
}
```

Các thành viên ảo của một lớp

- ❖ Trong C++, thành viên ảo (virtual member) thường là:
 - Hàm ảo (virtual function)
 - Hàm hủy ảo (virtual destructor)
- ❖ Hàm ảo cho phép đa hình động, tức là chương trình chọn hàm phù hợp trong lúc chạy, chứ không phải trong lúc biên dịch.
- ❖ Các thành viên khác (biến, hàm tĩnh, constructor) **không thể ảo**
- ❖ Cách hoạt động:
 - Khi một lớp có hàm ảo và lớp con ghi đè (override) hàm đó → chương trình sẽ gọi hàm của lớp con nếu đối tượng thực tế là lớp con.
 - Nếu không phải hàm ảo thì nó sẽ gọi theo kiểu con trỏ/tham chiếu.
 - Khi bạn xóa đối tượng bằng con trỏ lớp cha, **hàm hủy phải là ảo**, để đối tượng lớp con được giải phóng đúng cách và để tránh lỗi bộ nhớ.

Ví dụ - Gửi thông báo (20')

- ❖ Ứng dụng gửi thông báo bằng nhiều kênh guithongbao() cho mọi kênh, nhưng từng

```
#include <iostream>
#include <vector>
#include <memory> // unique_ptr
using namespace std;

// Lớp trừu tượng
class ThongBao {
public:
    virtual void guithongbao(const string& tieude, const string& noidung) = 0;
    virtual ~ThongBao() {} // destructor ảo
};

// Phương thức Email
class Email : public ThongBao {
    string diachi;
public:
    Email(const string& to) : diachi(to) {}
    void guithongbao(const string& tieude, const string& noidung) override {
        cout << "[Email] To: " << diachi << ", Tiêu đề: " << tieude
            << ", Nội dung: " << noidung << "\n";
    }
};
```

```
// Phương thức SMS
class SMS : public ThongBao {
    string sdt;
public:
    SMS(const string& s) : sdt(s) {}
    void guithongbao(const string& tieude, const string& noidung) override {
        cout << "[SMS] To: " << sdt << ", Nội dung: " << noidung << "\n";
    }
};

// Push
class Push : public ThongBao {
    string deviceId;
public:
    Push(const string& id) : deviceId(id) {}
    void guithongbao(const string& tieude, const string& noidung) override {
        cout << "[Push] Device: " << deviceId << ", Title: " << tieude << "\n";
    }
};

int main() {
    vector<unique_ptr<ThongBao>> ds;

    ds.push_back(unique_ptr<ThongBao>(new Email("hunglv@hvn.edu.vn")));
    ds.push_back(unique_ptr<ThongBao>(new SMS("0906238311")));
    ds.push_back(unique_ptr<ThongBao>(new Push("device-xyz")));

    for (size_t i = 0; i < ds.size(); i++) {
        ds[i]->guithongbao("Thông báo", "Lịch thi có thay đổi");
    }

    return 0;
}
```

Bài tập cuối chương

❖ Ứng dụng Grab cung cấp nhiều loại phương tiện để phục vụ khách hàng:

❖ **GrabBike:**

- ✓ Dành cho 1 khách.
- ✓ Giá rẻ, di chuyển nhanh.
- ✓ Phí: 8.000 VNĐ/km.

❖ **GrabCar:**

- ✓ Dành cho tối đa 4 khách.
- ✓ Tiện nghi hơn, phù hợp nhóm nhỏ.
- ✓ Phí: 12.000 VNĐ/km.

❖ **GrabCarPlus:**

- ✓ Dành cho tối đa 7 khách.
- ✓ Xe rộng rãi, thoải mái, thích hợp gia đình/nhóm đông người.
- ✓ Phí: 15.000 VNĐ/km.

Bài tập cuối chương

❖ Yêu cầu:

- Xây dựng **lớp trừu tượng** **PhuongTien** với các hàm thuần ảo:
 - ✓ `Hien_thi_thong_tin()`: Hiển thị thông tin phương tiện
 - ✓ `Tinhchiphi(double quangduong)`: Tính chi phí chuyển đi
- Xây dựng các lớp kế thừa:
 - ✓ `GrabBike`
 - ✓ `GrabCar`
 - ✓ `GrabCarPlus`
- Viết chương trình chính:
 - ✓ Người dùng nhập **số khách** và **quãng đường (km)**.
 - ✓ Hệ thống duyệt qua danh sách các phương tiện → tìm phương tiện phù hợp.
 - ✓ In ra thông tin phương tiện gợi ý cùng chi phí dự kiến.